Name: Arya Bailur	Class: T1	Roll No: 2401008
Batch: T11	Subject: Agile Software Development & DevOps	Deptartment: AI & DS

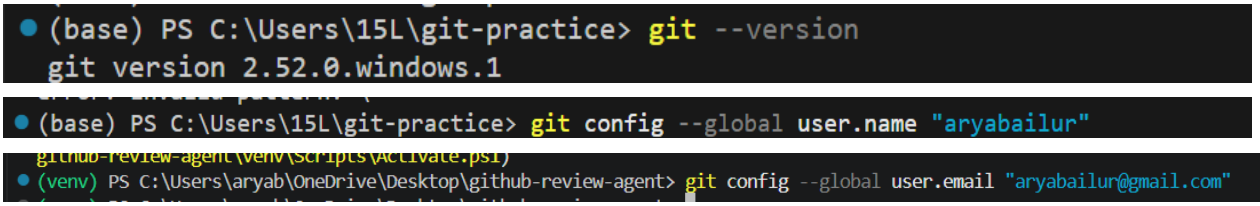
Aim: To perform version control operations using Git.

LO 02: Introduction to Git, GitHub, Basic operations: clone, commit, push, pull, Branching strategies and merging, Git workflows: Feature branching, Git Flow, Handling merge conflicts, GitHub Actions basics

Git Commands — Version Control Experiment

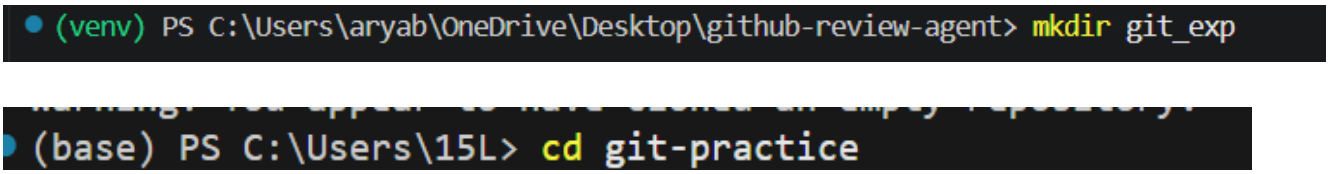
Setup & Configuration

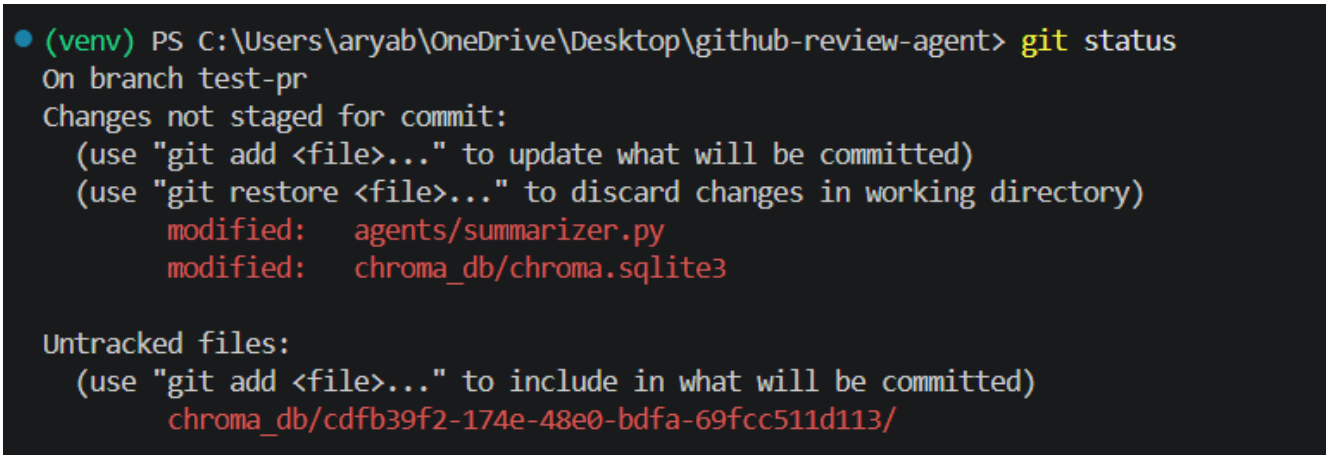
1. git --version — Checks which version of Git is installed on the system.
2. git config --global user.name “aryabailur" — Sets the username attached to your commits.
3. git config --global user.email “aryabailur@gmail.com” — Sets the email attached to your commits.



Creating the Repository

4. mkdir git_experiment — Creates a new folder for the project.
5. cd git_experiment — Moves into that folder.
6. git init — Initializes a new Git repository in the current folder.
7. git status — Shows the current state of the working directory (tracked/untracked/staged files).



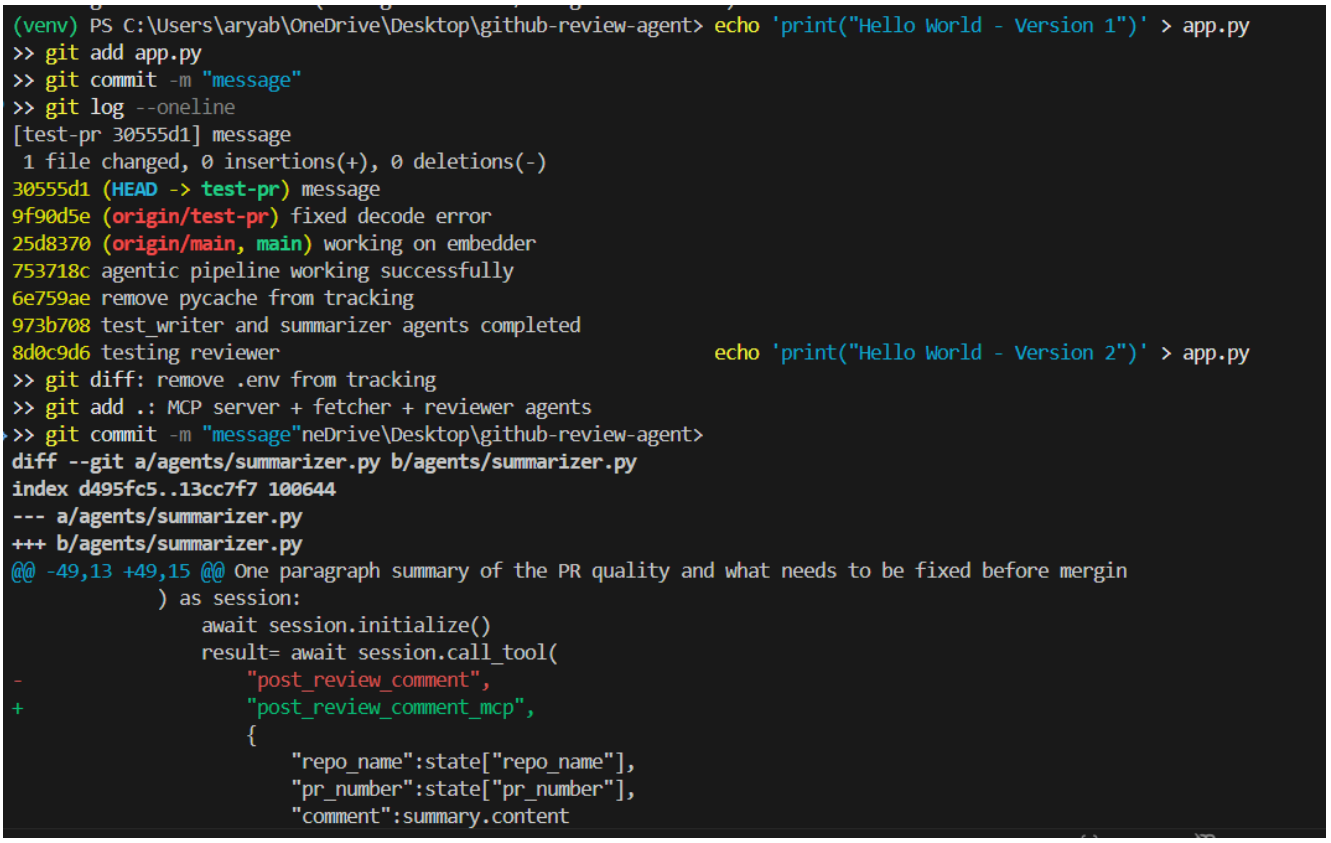


Creating & Committing Files

8. echo 'print("Hello World - Version 1")' > app.py — Creates a Python file with starter code.
9. git add app.py — Stages a specific file, marking it ready to be committed.
10. git commit -m "message" — Saves a permanent snapshot of staged changes with a message.
11. git log --oneline — Displays commit history in a short, one-line-per-commit format

Editing & Comparing Versions

12. echo 'print("Hello World - Version 2")' > app.py — Edits the file to create a new version.
13. git diff — Shows line-by-line differences between the working file and the last commit.
14. git add . — Stages all changed/new files in the folder at once.
15. git commit -m "message" — Commits the newly staged changes.



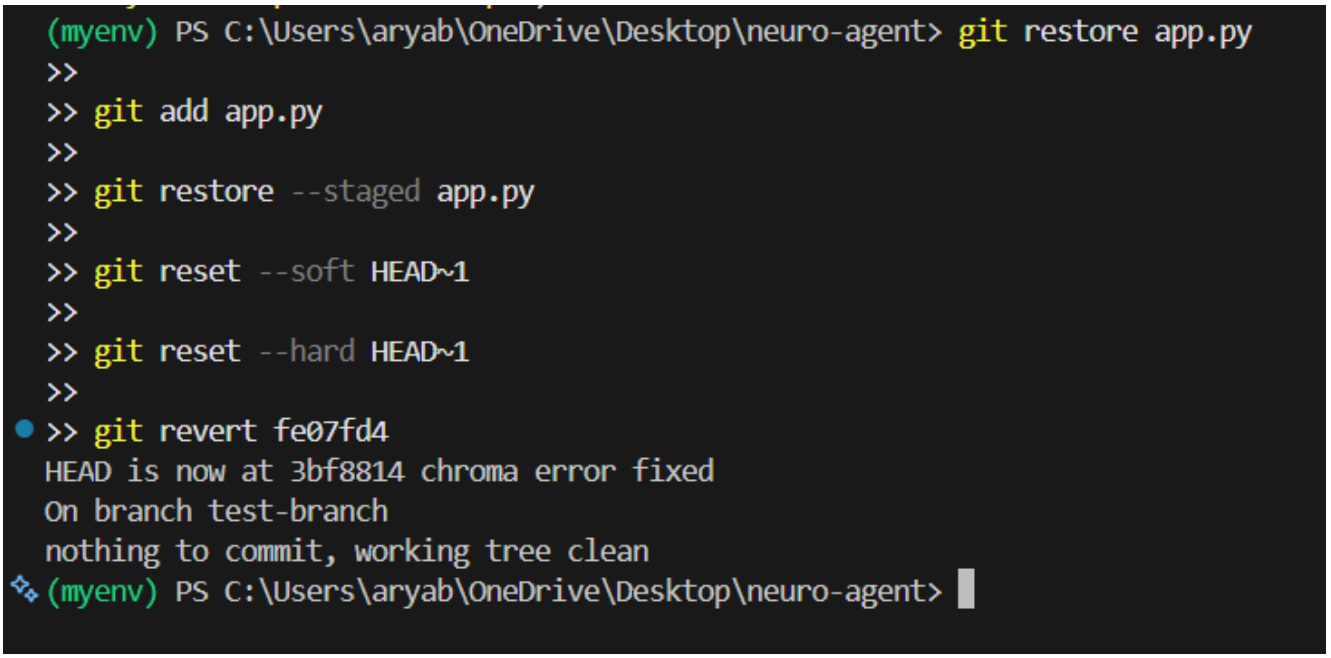
Branching

16. git branch — Lists all branches and shows which one is currently active.
17. git branch feature-branch — Creates a new branch without switching to it.
18. git checkout feature-branch — Switches the working directory to the specified branch.
19. echo 'print(...)' > app.py — Edits the file while on the new branch.
20. git commit -am "message" — Stages tracked files and commits in a single step.
21. git checkout master — Switches back to the main branch.
22. git merge feature-branch — Combines changes from feature-branch into the current branch.
23. git checkout -b test-branch — Creates and switches to a new branch in one command.

```
(myenv) PS C:\Users\aryab\OneDrive\Desktop\neuro-agent> git branch
>> git branch feature-branch
>> git checkout feature-branch
>> echo 'print("Hello from feature branch")' > app.py
>> git commit -am "Update app.py in feature branch"
>> git checkout master
>> git merge feature-branch
>> git checkout -b test-branch
* main
Switched to branch 'feature-branch'
[feature-branch fe07fd4] Update app.py in feature branch
 1 file changed, 0 insertions(+), 0 deletions(-)
error: pathspec 'master' did not match any file(s) known to git
Already up to date.
Switched to a new branch 'test-branch'
(myenv) PS C:\Users\aryab\OneDrive\Desktop\neuro-agent>
```

Undoing Changes / Rollback

24. git restore app.py — Discards uncommitted changes in a file, reverting it to the last commit.
25. git add app.py — Stages a file (used before demonstrating unstage below).
26. git restore --staged app.py — Unstages a file without deleting the actual edit.
27. git reset --soft HEAD~1 — Undoes the last commit but keeps changes staged.
28. git reset --hard HEAD~1 — Completely undoes the last commit and discards the changes (destructive).
29. git revert <commit-id> — Creates a new commit that undoes an older commit's changes, without deleting history (the safe rollback method).



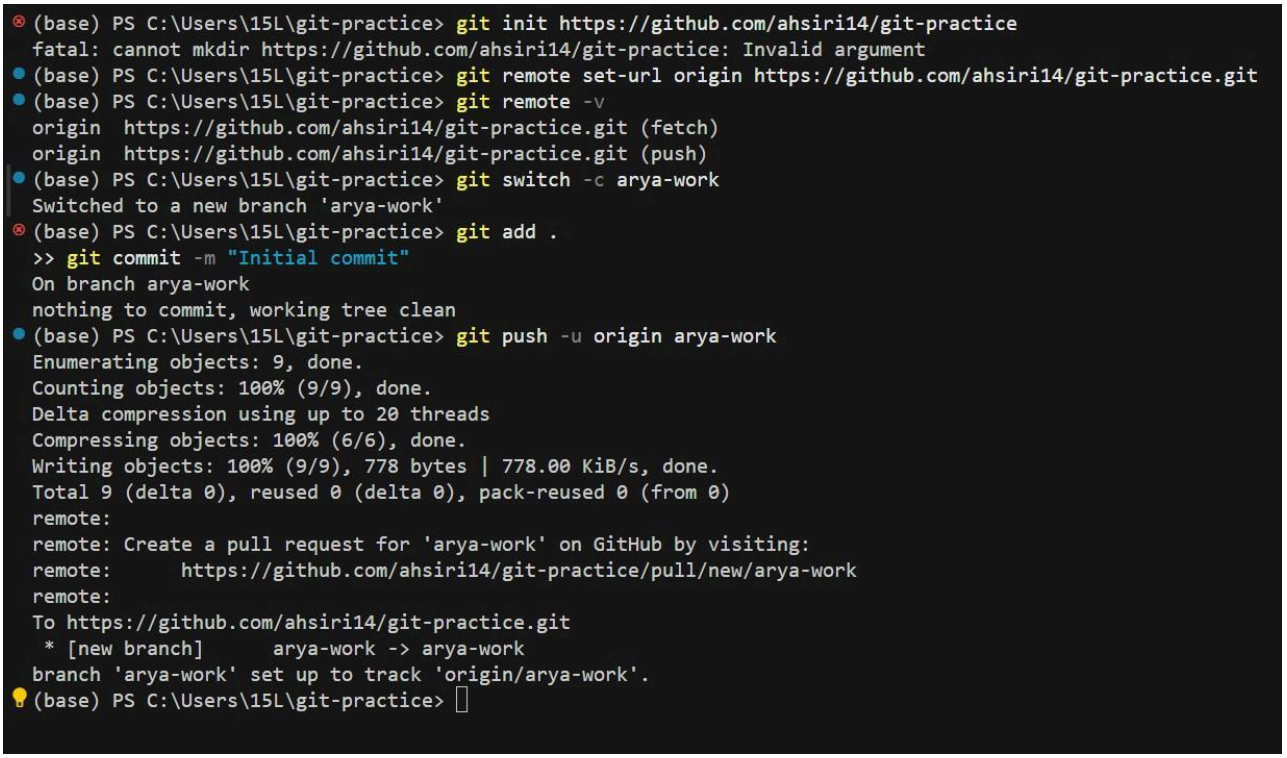
Viewing Final State

30. git log --oneline — Final check showing the complete commit history after all operations.

```
(myenv) PS C:\Users\aryab\OneDrive\Desktop\neuro-agent> git log --oneline
3bf8814 (HEAD -> test-branch, origin/main, main) chroma error fixed
b119a9d chroma error fixed
542224c chroma error fixed
6c42062 chroma error fixed
b4397f9 remove pycache from tracking
ae671df remove chroma_db from tracking
5003ab0 Update README.md
28accb6 Update requirements.txt
1a259b8 chroma env added
d1ef7e8 chroma env added
5e464cf improve arxiv client stability
03aa593 add arxiv dependency
8ae8347 add arxiv dependency
b229cec add chroma dependencies
ef0e3a5 add chroma dependencies
4bf3439 minimal requirements
0fea786 clean requirements
agent\vector_store.py ev Container Folder
57ee4d2 clean requirements
00da6a4 fixed requirements
39df8d6 README added
68b6939 Initial commit for Neuro-Tech repository
(myenv) PS C:\Users\aryab\OneDrive\Desktop\neuro-agent>
```

Collaboration

31. git remote set-url origin https://github.com/ahsiri14/git-practice -Changes the URL of the existing remote repository named origin
32. git remote -v - Displays the remote repositories configured for fetch and push operations.
33. git-practice (fetch)- Shows the remote URL Git uses to fetch changes from the repository.
34. git-practice (push)- Shows the remote URL Git uses to push changes to the repository.
35. git switch -c arya-work- Creates a new branch named and arya-work switches to it.
36. git push -u origin work- Pushes the arya-work branch to the remote repository and sets it as the default upstream branch.



Conclusion:This Git experiment demonstrated the basic version control workflow, including repository creation, file tracking, committing changes, branching, merging, reverting changes, and managing remote repositories. By performing these operations, we gained practical experience in using Git to track project history, collaborate efficiently, and safely manage different versions of code.